Universität Bremen

Praktische Informatik 3 WS 26/27

Christoph Lüth
Tobias Diehl
Marvin Hillmann
Marina Koksharova
Andreas Neumann
Tim Sperling

1. Übungsblatt

Ausgabe: 13.10.2026
Abgabe: 20.10.2026

*Dieses ist ein **Einzelübungsblatt**.*

*Bitte bearbeiten Sie **zwei** der folgenden Aufgaben.*

1.1 *Quersumme*

Die *Quersumme* einer natürlichen Zahl ist die Summe ihrer Ziffern. Wir können das rekursiv so berechnen (wobei $\text{quer}(x)$ die Quersumme der Zahl x sei, und $x \div y$ die ganzzahlige Division von x durch y):

$$\text{quer}(x) \stackrel{def}{=} \begin{cases} 0 & x = 0 \\ x \bmod 10 + \text{quer}(x \div 10) & \text{sonst} \end{cases} \tag{1}$$

Implementieren Sie eine Haskell-Funktion

```
quer :: Integer→ Integer
```

welche die Quersumme berechnet:

```
quer 5  ⇝  5
quer 457  ⇝  16
```

Sie können annehmen, dass das Argument eine nicht-negative ganze Zahl ist.

1.2 *Perfekt!*

Eine natürliche Zahl $n \in \mathbb{N}$ heißt *perfekt*, wenn die Summe der Teiler von n (einschließlich 1, aber ausschließlich n selber) genau wieder n ergibt. Ein Beispiel für eine perfekte Zahl ist 28: die Teiler sind $1+2+4+7+14 = 28$; ein Gegenbeispiel ist 45 mit $1+3+5+9+15 = 33$.

Wir wollen eine Funktion

```
perfect :: Integer→ Bool
```

implementieren, die prüft, ob eine Zahl perfekt ist. Dazu gehen wir in mehreren Schritten vor:

1. Zuerst schreiben wir eine Funktion `divides n m`, die prüft, ob `n` ein Teiler von `m` ist; das ist der Fall, wenn der Rest von `m` geteilt durch `n` 0 ist, d.h. $a \bmod b = 0$.

   ```
   divides :: Integer→ Integer→ Bool
   divides 3 7  ⇝  False
   divides 7 35  ⇝  True
   ```

2. Dann schreiben wir eine Funktion `sumOfDivisors`, welche die Summe der Teiler von `n` berechnet:

   ```
   sumOfDivisors :: Integer→ Integer
   sumOfDivisors 3  ⇝  1
   sumOfDivisors 45  ⇝  33
   sumOfDivisors 28  ⇝  28
   ```

   Dazu müssen wir alle Zahlen von `1` bis `n-1` prüfen, ob sie ein Teiler von `n` sind; ist das der Fall, werden sie zum Ergebnis addiert, ansonsten nicht. Das machen wir, indem wir eine Hilfsfunktion `sumDiv2 n m` implementieren, wobei `m` (rekursiv!) die Werte von `1` bis `n-1` durchläuft.

3. Mit `sumOfDivisors` können wir jetzt die gewünschte Funktion `perfect` leicht implementieren.

Die ersten perfekten Zahlen sind übrigens $6, 28, 496$ und 8128. Es ist unbekannt, wie viele perfekte Zahlen es gibt, oder ob es auch ungerade perfekte Zahlen gibt.[1] Im Gegensatz zu Primzahlen haben sich für perfekte Zahlen noch keine Anwendungen in der Kryptologie gefunden, aber weil sie vergleichsweise selten sind, werden sie unter Kennern zu Höchstpreisen als NFTs gehandelt.

1.3 *Fermats Testament*

Primzahlen sind immer nützlich, mit Rucola und Parmesan als leichter Salat oder zur Schlüsselgenerierung für kryptographische Verfahren. Aber wann ist eine Zahl eine Primzahl? Dazu wollen wir in dieser Übung Verfahren implementieren.

1. Schreiben Sie eine Funktion

   ```
   slowprime :: Integer→ Bool
   ```

   die prüft, ob das Argument `n` eine Primzahl ist, indem es für alle Zahlen von `2` bis $\lfloor\sqrt{n}\rfloor$ prüft, ob n durch diese Zahl teilbar ist. ($\lfloor\sqrt{n}\rfloor$ ist die *ganzzahlige Wurzel* aus n, und ist mit dem Namen `isqrt` in der Vorlage zum Aufgabenblatt vorgegeben). Hierbei ist a durch b teilbar, wenn der Rest der ganzzahligen Division 0 ist (vgl. vorherige Aufgabe).

   Hinweis: Beachten Sie bitte: 1 ist *keine* Primzahl.

2. Ihr Nachbar gibt an, dass sein Urururgroßvater der berühmte Mathematiker Pierre de Fermat war. Er (der Mathematiker, nicht ihr Nachbar) wurde berühmt, weil er der Nachwelt mit einer fadenscheinigen Begründung ("Kein Platz mehr es aufzuschreiben…") ein Theorem ohne Beweis hinterließ. Zum Glück hat Monsieur de Fermat einige seiner Theoreme tatsächlich bewiesen, darunter das kleine Fermatsche Theorem: wenn p eine Primzahl ist, dann gilt für alle mit p teilerfremden a

   $$a^{p-1} \bmod p = 1 \tag{2}$$

   Ihr Nachbar sagt jetzt, man müsse doch einfach nur prüfen, ob die Gleichung (2) für $a = 2$ gilt, das sei doch viel schneller als dieses ganze Gezähle. Ist dies nicht der Fall, kann p keine Primzahl sein, aber stimmt das auch andersherum — d.h. wenn die Gleichung gilt, ist p dann auch eine Primzahl?

   Schreiben Sie eine Funktion

   ```
   fastprime :: Integer→ Bool
   ```

   die nach dem Fermatschen Verfahren prüft, ob `n` eine Primzahl ist, d.h. ob $2^{(n-1)} \bmod n = 1$ gilt.

3. Jetzt wird sich zeigen, ob Ihr Nachbar recht hat. Schreiben Sie eine Funktion

   ```
   count_false :: Integer→ Integer
   ```

   die zählt, für wieviele Zahlen von $1 \ldots n$ der Fermatsche Test ein falsches Ergebnis liefert.

   Hinweis: Ihr Nachbar hat *nicht* recht (er ist auch sonst ein ziemlicher Angeber, hat zu allem eine Meinung aber von nichts viel Ahnung und einen HSV-Sticker an seinem SUV): von $1 \ldots 100$ gibt es eine, und für $1 \ldots 1000$ vier Abweichungen (welche?).[2]

Sie können annehmen, dass die Argumente Ihrer Primzahltests nicht negativ sind.

1.4 *Hexadezimales*

Implementieren sie zwei Funktionen

```
encode :: Integer→ String
decode :: String→ Integer
```

die eine ganze Zahl in *Hexadezimaldarstellung* ausgeben beziehungsweise einlesen. Die Ziffern von 10 bis 15 sollen wie üblich durch die Buchstaben `a` bis `f` dargestellt werden:

[1] `https://www.britannica.com/science/perfect-number`

[2] Zur Ehrenrettung von Fermat sei gesagt, dass es auf diesem Test aufbauende Verfahren wie den Miller-Rabin-Test gibt, die wesentlich robuster sind.

```
*Hex> encode 65347
"ff43"
*Hex> decode "ff43"
65347
```

Hinweise:

1. Die Funktion `isHexDigit :: Char→ Bool` prüft, ob ein Zeichen eine hexadezimale Ziffer ist.
2. Die Funktion `digitToInt :: Char→ Int` wandelt eine hexadezimale Ziffer in eine ganze Zahl um, und die Funktion `intToDigit :: Int→ Char` wandelt eine ganze Zahl in eine hexadezimale Ziffer um.

1.5 *Matrikelnummern*

Die Matrikelnummer ist der Studierenden treuer Begleiter durch alle Höhen und Tiefen der Uni-Zeit. Aber nicht jede Zahl ist eine Matrikelnummer, sondern diese besteht aus einer siebenstelligen Zahl mit einer Prüfziffer am Ende. Wie wird diese Prüfziffer berechnet? In dieser Aufgaben lüften wir das Geheimnis.

Die Matrikelnummer ist siebenstellig, mit einer sechsstelligen Zahl plus einer Prüfziffer, die wir wie folgt berechnen: aus der sechsstelligen Zahl wird eine gewichtete Quersumme gebildet, bei der jede Ziffer mit der Stelle (Index von links gezählt und bei 1 beginnend) multipliziert wird. Die Prüfziffer ist dann der Rest (Modulus) mit 11, wobei der Rest 10 der 0 entspricht. Hier eine Beispielrechnung für die Zahl 325687:

Ziffern der Ausgangszahl	3	2	5	6	8	7
Index	1	2	3	4	5	6
Ziffer · Index	3	4	15	24	40	42
Gewichtete Summe			128			

Als Summe ergibt sich 128, die Prüfziffer ist damit 7.

Schreiben Sie eine Funktion, welche für eine beliebige positive Zahl (muss nicht unbedingt sechsstellig sein) die Prüfziffer berechnet:

```
pruefziffer :: Integer→ Int
pruefziffer 325687 ⇝ 7
pruefziffer 378843 ⇝ 1
pruefziffer 235658 ⇝ 0
```

Implementieren Sie damit eine Funktion, welche für eine siebenstellige positive ganze Zahl prüft, ob diese eine Matrikelnummer sein kann:

```
checkMatrNo :: Integer→ Bool
checkMatrNo 6256870 ⇝ True
checkMatrNo 1574227 ⇝ False
```

Hinweise:

1. Die Rekursion für die Prüfziffer ist ähnlich wie diejenige für die Quersumme, aber Sie sollten sich eine Hilfsfunktion definieren, welche die Stelle als zusätzlichen Parameter mitzählt.
2. Da die Rekursion mit der kleinsten Stelle anfängt, die aber den höchsten Index hat, benötigen Sie zusätzlich eine Funktion, welche die Stelligkeit der Zahl berechnet.
3. Alternativ können Sie auch die Zeichenkette in eine String konvertieren, und über diesen iterieren.